

项目报告

文件管理系统

学院： 网络与通讯学院

专业： 计算机网络技术

年级： 计网 25-2

组长： 乐陈怡

成员： 乐陈怡，李震东

何锐，范家瑞，肖涵

目录

一、项目内容进程安排	4
二、知识总结	4
2.1 主要知识点	4
2.1.1 文件路径处理与工作目录管理	4
2.1.2 文件和目录的基本操作	5
2.1.3 JSON 数据持久化与回收站元数据管理	6
2.1.4 时间戳命名与同名文件冲突处理	7
2.1.5 全局导航机制与哨兵对象	8
2.1.6 状态机思想与多步骤交互	8
2.1.7 递归遍历与目录树展示	9
2.1.8 文件搜索与通配符匹配	10
2.1.9 文件统计与系统信息获取	10
2.1.10 异常处理与程序健壮性	11
2.2 重点	11
2.2.1 统一输入与全局导航设计	11
2.2.2 回收站软删除机制	12
2.2.3 多步骤功能的状态机控制	12
2.2.4 递归处理能力	13
2.2.5 元数据一致性维护	13
2.3 难点	13
2.3.1 难点一：全局导航在多层函数中的传递	13
2.3.2 难点二：返回上一步的状态恢复	14
2.3.3 难点三：导航快捷键与真实输入冲突	14
2.3.4 难点四：回收站文件名冲突与还原路径冲突	14
2.3.5 难点五：文件内容搜索的兼容性	15
三、项目实施	15
3.1 系统总体架构	15
3.2 全局导航与工具模块实现	16
3.2.1 BACK 和 MAIN_MENU	16
3.2.2 safe_input(prompt)	16
3.2.3 input_multiline(prompt)	17
3.2.4 ensure_recycle_bin()	17
3.2.5 load_recycle_meta()和 save_recycle_meta(meta)	18
3.2.6 get_timestamp()	18
3.3 文件操作模块实现	18
3.3.1 create_file()	18
3.3.2 delete_file()	19
3.3.3 rename_file()	19
3.3.4 copy_move_file()	20
3.4 目录操作模块实现	21
3.4.1 create_directory()	21

3.4.2 delete_directory()	21
3.4.3 traverse_directory()	22
3.5 搜索与统计模块实现	22
3.5.1 file_search()	22
3.5.2 file_statistics()	23
3.6 回收站管理模块实现	24
3.6.1 recycle_bin_menu()	24
3.6.2 list_recycle_bin()	24
3.6.3 restore_from_recycle()	25
3.6.4 empty_recycle_bin()	25
3.6.5 permanent_delete_from_recycle()	26
3.7 主菜单调度模块实现	26
3.7.1 main()	26
3.8 学习心得	27

一. 项目内容进程安排

表 1.1 项目内容及进程安排

时间段	学习、工作安排	完成情况
第 15 周第 1 次课	需求分析与系统设计，确定全局导航架构和核心功能模块	已完成
第 15 周第 2 次课	核心功能实现：文件 / 目录创建、删除、重命名、复制移动	已完成
第 16 周第 1 次课	扩展功能实现：目录树遍历、文件内容搜索、统计及回收站增强	已完成
第 16 周第 2 次课	系统测试与 bug 修复，完善全局导航逻辑，编写项目报告	已完成

二、知识总结

2.1 主要知识点

2.1.1 文件路径处理与工作目录管理

文件管理系统首先需要解决“用户输入的路径如何定位”的问题。用户可能输入相对路径，例如 `test.txt`，也可能输入绝对路径，例如 `C:\Users\...\test.txt`。因此程序通过 `os.path.isabs()` 判断输入路径是否为绝对路径。如果不是绝对路径，就使用 `os.path.join(current_dir, name)` 将其拼接到当前工作目录下；如果是绝对路径，则直接使用用户输入的路径。

本项目使用全局变量 `current_dir` 保存当前工作目录，初始值由 `os.getcwd()` 获取。用户可以通过主菜单中的“切换工作目录”功能修改该变量。这样，各个文件操作函数都可以基于同一个工作目录运行，避免每个功能重复询问完整路径。

相关知识点：

知识点	项目中作用
<code>os.getcwd()</code>	获取程序启动时的当前目录
<code>os.path.isabs()</code>	判断用户输入是否为绝对路径
<code>os.path.join()</code>	将当前目录和相对路径拼接
<code>os.path.exists()</code>	判断文件或目录是否存在
<code>os.path.abspath()</code>	将路径转换为绝对路径
<code>os.path.isdir()/os.path.isfile()</code>	区分目录和文件

对应函数：

`create_file()`、`delete_file()`、`rename_file()`、`copy_move_file()`、
`create_directory()`、`delete_directory()`、`traverse_directory()`、
`file_search()`、`file_statistics()`、`main()`

2.1.2 文件和目录的基本操作

本项目的核心功能是文件和目录管理，因此大量使用 `os` 和 `shutil` 两个标准库。`os` 更适合处理路径判断、目录创建、重命名、文件属性获取等基础操作；`shutil` 更适合进行复制、移动、递归删除等高级文件操作。

在创建文件时，程序先使用 `os.makedirs()` 确保目标文件的父目录存在，再通过 `open(path, "w", encoding="utf-8")` 写入内容。在重命名功能中，使用 `os.rename()` 修改文件或目录名称。在复制和移动功能中，程序根据源路径类型选择不同方法：如果源路径是目录，则复制时使用 `shutil.copytree()`；如果源路径是普通文件，则使用 `shutil.copy2()`，这样可以尽量保留文件的元数据；移动时统一使用 `shutil.move()`。

相关知识点：

知识点	项目中作用
<code>open()</code>	创建文件、写入内容、读取文件内容
<code>os.makedirs()</code>	创建目录或确保父目录存在
<code>os.rename()</code>	实现文件或目录重命名
<code>shutil.copy2()</code>	复制文件并保留部分元数据

<code>shutil.copytree()</code>	递归复制整个目录
<code>shutil.move()</code>	移动文件、移动目录、实现回收站软删除
<code>shutil.rmtree()</code>	永久删除回收站中的目录
<code>os.remove()</code>	永久删除回收站中的文件

对应函数：

`create_file()`、`rename_file()`、`copy_move_file()`、`create_directory()`、
`delete_file()`、`delete_directory()`、`empty_recycle_bin()`、
`permanent_delete_from_recycle()`

2.1.3 JSON 数据持久化与回收站元数据管理

普通删除操作一旦执行，文件可能无法恢复。为了提高安全性，本项目设计了“回收站机制”：删除文件或目录时，不直接永久删除，而是将其移动到用户主目录下的`.recycle_bin`文件夹中，同时使用`recycle_meta.json`保存删除记录。

回收站元数据采用 JSON 格式保存，每一条记录包含三个字段：

```
{  
  "original_path": "文件或目录的原始路径",  
  "deleted_at": "删除时间",  
  "bin_name": "回收站中的文件名"  
}
```

使用 JSON 的好处是结构清晰，便于程序读取和修改。`load_recycle_meta()` 使用 `json.load()` 将 JSON 文件转换为 Python 列表；`save_recycle_meta(meta)` 使用 `json.dump()` 将更新后的列表写回文件。项目中还设置了 `ensure_ascii=False`，确保中文路径不会被转义成难以阅读的 Unicode 编码。

相关知识点:

知识点	项目中作用
<code>json.load()</code>	读取回收站元数据
<code>json.dump()</code>	保存回收站元数据
<code>ensure_ascii=False</code>	正确保存中文路径
<code>indent=2</code>	让 JSON 文件更易阅读
字典和列表	组织每一条删除记录

对应函数:

`ensure_recycle_bin()`、`load_recycle_meta()`、`save_recycle_meta()`、
`delete_file()`、`delete_directory()`、`list_recycle_bin()`、
`restore_from_recycle()`、`empty_recycle_bin()`、
`permanent_delete_from_recycle()`

2.1.4 时间戳命名与同名文件冲突处理

如果用户多次删除同名文件，直接放入回收站会产生覆盖风险。为了解决该问题，项目通过 `datetime.datetime.now().strftime("%Y%m%d_%H%M%S")` 生成时间戳，并将时间戳拼接到回收站文件名中。

例如，用户删除 `test.txt` 时，回收站中的文件名可能变为：

```
test_20260615_143025.txt
```

这样即使用户多次删除同名文件，回收站中也能保存不同版本。同时，元数据中 `original_path` 仍然记录文件原来的位置，还原时不依赖回收站文件名猜测路径，而是直接根据元数据恢复。

相关知识点:

知识点	项目中作用
<code>datetime.datetime.now()</code>	获取当前时间
<code>strftime()</code>	将时间格式化为字符串
<code>os.path.splitext()</code>	分离文件名和扩展名
<code>os.path.basename()</code>	获取路径中的文件名

对应函数：

`get_timestamp()`、`delete_file()`、`delete_directory()`

2.1.5 全局导航机制与哨兵对象

命令行程序如果每一步输入错误都只能重新运行，会影响使用体验。因此项目设计了全局导航机制：在大多数输入界面中，用户输入 **0** 可以返回主菜单，输入 **b** 可以返回上一步。

为了避免把普通字符串和导航命令混淆，项目定义了两个特殊对象：

```
BACK = object()
MAIN_MENU = object()
```

这两个对象不是字符串，因此不会和用户正常输入的文件名、路径、关键词冲突。`safe_input(prompt)` 负责统一处理用户输入：输入 **0** 返回 `MAIN_MENU`，输入 **b** 返回 `BACK`，其他内容正常返回字符串。后续版本还增加了 `\0` 和 `\b` 转义规则，允许用户输入字面值 **0** 或 **b**，解决了导航快捷键与真实输入内容冲突的问题。

相关知识点：

知识点	项目中作用
函数封装	将重复输入逻辑统一到 <code>safe_input()</code>
<code>object()</code>	创建唯一的哨兵对象
返回值分支判断	根据 <code>BACK</code> 和 <code>MAIN_MENU</code> 控制流程
转义输入	使用 <code>\0</code> 、 <code>\b</code> 输入字面值

对应函数：

`safe_input()`、`input_multiline()`、`create_file()`、`copy_move_file()`、`file_search()`、`recycle_bin_menu()`、`restore_from_recycle()`、`main()`

2.1.6 状态机思想与多步骤交互

部分功能不是一步完成的，例如复制或移动文件需要经历“选择复制/移动方式、输入源路径、输入目标路径”三个步骤；文件搜索需要经历“选择搜索方式、输入搜索目录、输入搜索模式或关键词”三个步骤。如果使用简单的顺序结

构，用户输入错误后很难返回上一步。

项目通过 **step 变量** 实现简单状态机。每个步骤对应一个数字，例如 `step = 1` 表示选择操作类型，`step = 2` 表示输入源路径，`step = 3` 表示输入目标路径。当用户输入 **b** 时，程序将 **step** 减小或设置为上一个步骤；当用户输入 **0** 时，直接返回主菜单。

这种设计让命令行程序具有更清晰的交互流程，也降低了误操作后的重新输入成本。

对应函数：

`copy_move_file()`、`file_search()`、`permanent_delete_from_recycle()`

2.1.7 递归遍历与目录树展示

目录管理功能需要递归处理多级目录。项目使 `os.walk()` 遍历目录树。`os.walk(path)` 每次返回当前目录路径、子目录列表和文件列表，因此不仅可以用于目录树展示，也可以用于文件搜索和目录统计。

在 `traverse_directory()` 中，程序通过计算当前目录相对根目录的层级，生成对应数量的缩进符号，再输出类似 `tree` 命令的结构：

```
当前目录/
|   ├── file1.txt
|   ├── file2.py
|   └── subdir/
```

相关知识点：

知识点	项目中作用
<code>os.walk()</code>	递归遍历目录
<code>os.sep</code>	识别系统路径分隔符
字符串重复	根据层级生成缩进
循环嵌套	使用 <code>\0</code> 、 <code>\b</code> 输入字面值

对应函数：

`traverse_directory()`、`file_search()`、`file_statistics()`

2.1.8 文件搜索与通配符匹配

项目提供两种搜索方式：按文件名搜索和按文件内容搜索。按文件名搜索时，用户可以输入类似`*.txt`、`*.py`的通配符模式，程序使用 `glob.fnmatch.fnmatch()` 判断文件名是否符合模式。按内容搜索时，程序递归遍历目录下的所有文件，逐行读取文本内容，如果某一行包含关键词，就输出文件路径、行号和内容片段。

内容搜索涉及文件编码问题。为了避免部分文件因编码不兼容导致程序崩溃，读取文件时使用 `encoding="utf-8"`，`errors="ignore"`。对于权限不足或无法打开的文件，程序通过异常处理跳过，继续搜索其他文件。

相关知识点：

知识点	项目中作用
通配符匹配	支持 <code>*.txt</code> 等文件名搜索
逐行读取	搜索文件内容并定位行号
<code>enumerate(f, 1)</code>	输出匹配内容所在行号
<code>errors="ignore"</code>	忽略无法解码的字符

对应函数：

`file_search()`

2.1.9 文件统计与系统信息获取

文件统计功能通过 `os.stat()` 获取文件大小、创建时间、修改时间和访问时间。对于普通文件，程序会显示扩展名；如果文件属于文本类型，还会统计行数。对于目录，程序使用 `os.walk()` 递归统计文件数量、子目录数量和总大小。

相关知识点：

知识点	项目中作用
<code>os.stat()</code>	获取文件元信息
<code>st_size</code>	获取文件大小
<code>st_ctime/st_mtime/st_atime</code>	获取创建、修改、访问时间
<code>datetime.fromtimestamp()</code>	将时间戳转为可读时间

```
sum(1 for _ in f)
```

统计文本文件行数

对应函数：

```
file_statistics()
```

2.1.10 异常处理与程序健壮性

文件系统操作容易遇到各种异常，例如路径不存在、权限不足、目标文件已存在、文件正在被占用、目录无法读取、用户中断操作等。如果不处理这些情况，程序容易直接崩溃。

本项目在文件创建、删除、复制、移动、遍历、搜索、统计和回收站管理中都使用了 `try-except` 结构。普通异常会被捕获并输出错误原因；对于耗时较长的目录遍历和搜索，程序还捕获 `KeyboardInterrupt`，允许用户中断操作后返回菜单。

对应函数：

```
create_file()、delete_file()、rename_file()、copy_move_file()、  
create_directory()、delete_directory()、traverse_directory()、  
file_search()、file_statistics()、restore_from_recycle()、  
empty_recycle_bin()、permanent_delete_from_recycle()、main()
```

2.2 重点

2.2.1 统一输入与全局导航设计

本项目的重点是统一输入逻辑。传统命令行程序往往每个功能单独写 `input()`，这样容易造成提示不统一，也很难实现“任意位置返回主菜单或上一步”的功能。项目将输入操作封装为 `safe_input(prompt)`，所有功能函数都通过它获取用户输入。

该设计的价值体现在两个方面：

① `0` 和 `b` 的导航行为在各功能中统一。

② 后续新增 `\0` 和 `\b` 转义规则时，只需要修改 `safe_input()`，其他函数可以继续复用。

2.2.2 回收站软删除机制

删除功能是文件管理系统中风险最高的功能。

删除文件或目录时，程序执行以下步骤：

- ①检查目标路径是否存在。
- ②判断目标是文件还是目录，避免用户用错功能。
- ③创建或检查回收站目录。
- ④生成时间戳文件名，避免同名覆盖。
- ⑤使用 `shutil.move()` 将目标移动到回收站。
- ⑥将原始路径、删除时间、回收站文件名写入 JSON 元数据。

这种设计将“删除”变成“可恢复的移动”，提高了程序的安全性。

2.2.3 多步骤功能的状态机控制

复制/移动、文件搜索、永久删除等功能都不是单步操作。如果用户在中间步骤想返回，程序必须知道当前处于哪一步。项目使用 `step` 变量控制流程，把多步骤功能拆成多个状态。

例如 `copy_move_file()` 中：

```
step 1: 选择复制或移动  
step 2: 输入源路径  
step 3: 输入目标路径
```

当用户输入 `b` 时，程序返回上一个 `step`；当用户输入 `0` 时，直接返回主菜单。

2.2.4 递归处理能力

项目中多个功能都依赖递归遍历：

功能	递归处理方式
目录遍历	使用 <code>os.walk()</code> 打印目录树
文件搜索	获取文件大小
目录统计	使用 <code>os.walk()</code> 统计文件数、目录数和总大小
目录复制	使用 <code>shutil.copytree()</code> 复制整个目录树
永久删除目录	使用 <code>shutil.rmtree()</code> 删除整个目录

2.2.5 元数据一致性维护

回收站功能不仅要移动文件，还要保证 JSON 元数据和实际回收站内容保持一致。例如，删除时需要新增元数据；还原时需要移除对应元数据；清空回收站时需要清空 JSON；永久删除某个条目时需要删除对应文件并从列表中移除该记录。

如果元数据没有同步更新，就可能出现“文件已经不存在但列表还显示”或“文件存在但无法还原”的问题。因此，元数据一致性是回收站模块的重点。

2.3 难点

2.3.1 难点一：全局导航在多层函数中的传递

项目中有主菜单、回收站子菜单、还原功能、永久删除功能等多层调用关系。用户在深层功能中输入 0 时，程序不能只返回上一层，而应该直接回到主菜单。

解决方案：

项目使用 `MAIN_MENU` 作为特殊返回值。当深层函数检测到用户输入 0 后，返回 `MAIN_MENU`；上层函数收到该值后继续向外返回，直到主菜单接管流程。这样实现了“导航穿透”。

典型函数：

`safe_input()`、`recycle_bin_menu()`、`restore_from_recycle()`、

`empty_recycle_bin()`、`permanent_delete_from_recycle()`

2.3.2 难点二：返回上一步的状态恢复

复制/移动和文件搜索都包含多个步骤。如果用户在第三步输入 `b`，程序应该回到第二步，而不是直接退出功能。单纯用顺序代码很难实现这种流程。

解决方案：

项目使用 `step` 变量记录当前步骤，并用 `while True` 循环包裹整个流程。不同 `step` 对应不同输入逻辑，输入 `b` 时修改 `step` 的值回到上一步。

典型函数：

`copy_move_file()`、`file_search()`、`permanent_delete_from_recycle()`

2.3.3 难点三：导航快捷键与真实输入冲突

项目规定输入 `0` 返回主菜单，输入 `b` 返回上一步。但在真实场景中，用户可能确实想创建名为 `0` 的文件，或者搜索关键词 `b`。如果不处理这个问题，导航快捷键会占用正常输入内容。

解决方案：

后续版本增加了转义输入规则：输入 `\0` 表示字面值“`0`”，输入 `\b` 表示字面值“`b`”。这样既保留了快捷导航，又允许用户输入特殊内容。

典型函数：

`safe_input()`

2.3.4 难点四：回收站文件名冲突与还原路径冲突

如果多个同名文件被删除，回收站中可能出现同名覆盖。另外，如果还原时原路径已经存在新文件，直接还原会覆盖用户现有文件。

解决方案：

删除时使用时间戳生成回收站文件名，避免回收站内部同名冲突。还原时先判断原始路径是否已经存在，如果存在则拒绝还原并提示用户，避免覆盖现有文件。

典型函数：

`get_timestamp()`、`delete_file()`、`delete_directory()`、
`restore_from_recycle()`

2.3.5 难点五：文件内容搜索的兼容性

目录遍历、文件搜索和目录统计可能处理大量文件，运行时间较长。如果用户想中途停止，直接按 Ctrl+C 可能导致程序退出。

解决方案：

项目在相关循环外捕获 `KeyboardInterrupt`，当用户中断时输出提示并返回菜单，而不是让程序崩溃。

典型函数：

`traverse_directory()`、`file_search()`、`file_statistics()`、`main()`

三、项目实施

3.1 系统总体架构

模块	主要职责	相关函数
全局导航与工具模块	处理输入、时间戳、回收站初始化、元数据读写	<code>safe_input()</code> 、 <code>input_multiline()</code> 、 <code>get_timestamp()</code> 、 <code>ensure_recycle_bin()</code> 、 <code>load_recycle_meta()</code> 、 <code>save_recycle_meta()</code>
文件操作模块	创建、删除、重命名、复制、移动文件	<code>create_file()</code> 、 <code>delete_file()</code> 、 <code>rename_file()</code> 、 <code>copy_move_file()</code>
目录操作模块	创建目录、删除目录、遍历目录树	<code>create_directory()</code> 、 <code>delete_directory()</code> 、 <code>traverse_directory()</code>
搜索统计模块	按文件名或内容搜索、统计文	<code>file_search()</code> 、 <code>file_statistics()</code>

	件和目录信息	
回收站 管理模 块	查看、还原、清 空、永久删除回 收站内容	recycle_bin_menu()、list_recycle_bin()、 restore_from_recycle()、empty_recycle_bin()、 permanent_delete_from_recycle()
主菜单 调度模 块	显示菜单、分发 用户选择、切换 工作目录	main()

系统入口为：

```
if __name__ == "__main__":
    main()
```

程序启动后进入 `main()` 主循环，用户根据菜单输入功能编号。主菜单根据用户选择调用对应功能函数，功能执行完毕后返回主菜单继续等待下一次操作。

3.2 全局导航与工具模块实现

3.2.1 BACK 和 MAIN_MENU

```
BACK = object()
MAIN_MENU = object()
```

这两个变量用于表示特殊导航命令。使用 `object()` 的原因是它们具有唯一性，不会和用户输入的字符串混淆。

3.2.2 safe_input(prompt)

该函数是全局导航系统的核心。它在普通 `input()` 的基础上增加了统一导航规则。

实现流程:

1. 显示输入提示, 并附加 (0:主菜单, b:上一步)。
2. 获取用户输入并去除首尾空格。
3. 如果输入为 0, 返回 MAIN_MENU。
4. 如果输入为 b, 返回 BACK。
5. 如果输入为 \0, 返回普通字符串 "0"。
6. 如果输入为 \b, 返回普通字符串 "b"。
7. 其他情况返回用户输入的普通字符串。

3.2.3 input_multiline(prompt)

该函数用于创建文件时输入多行内容。普通 `safe_input()` 适合单行输入, 而文件内容可能包含多行, 因此单独设计 `input_multiline()`。

实现流程:

1. 打印输入说明。
2. 循环读取每一行内容。
3. 输入空行时结束输入。
4. 输入 --menu 返回主菜单。
5. 输入 --back 返回上一步。
6. 使用 `"\n".join(lines)` 将多行内容拼接成完整文本。

3.2.4 ensure_recycle_bin()

该函数用于保证回收站目录和元数据文件存在, 保证后续读写回收站时不会因为目录或文件不存在而失败。

实现流程:

1. 判断 RECYCLE_BIN 是否存在。
2. 如果不存在, 使用 `os.makedirs()` 创建。
3. 判断 RECYCLE_META 是否存在。
4. 如果不存在, 创建 JSON 文件并写入空列表 []。

3.2.5 load_recycle_meta() 和 save_recycle_meta(meta)

这两个函数用于读写回收站元数据

实现流程:

load_recycle_meta():

1. 先调用 ensure_recycle_bin()。
2. 打开 recycle_meta.json。
3. 使 json.load() 返回元数据列表。

save_recycle_meta(meta):

4. 先调用 ensure_recycle_bin()。
5. 打开 recycle_meta.json。
6. 使用 json.dump() 保存元数据。
7. 设置 ensure_ascii=False 支持中文路径, 设置 indent=2 提高可读性。

3.2.6 get_timestamp()

该函数返回当前时间字符串, 格式为:



YYYYMMDD_HHMMSS

主要用于生成回收站文件名, 防止同名文件互相覆盖。

3.3 文件操作模块实现

3.3.1 create_file()

创建一个新的文本文件, 并允许用户输入多行文件内容。

实现流程:

1. 使用 safe_input() 获取文件名。
2. 如果用户输入 0 或 b, 返回主菜单。
3. 判断文件名是否为空。
4. 判断路径是相对路径还是绝对路径。
5. 如果文件已存在, 则提示错误并重新输入。

6. 调用 `input_multiline()` 获取文件内容。
7. 如果用户在内容输入阶段输入 `--back`，返回文件名输入步骤。
8. 使用 `os.makedirs()` 创建父目录。
9. 使用 `open(path, "w", encoding="utf-8")` 写入内容。
10. 使用 `try-except` 捕获创建失败异常。

实现原理：

该函数将“路径处理、重复检查、内容输入、文件写入”组合起来，完成文件创建流程。父目录自动创建可以让用户直接输入类似 `docs/test.txt` 的路径。

3.3.2 delete_file()

删除文件，但不是永久删除，而是移动到回收站。

实现流程：

1. 输入要删除的文件名。
2. 根据 `current_dir` 解析路径。
3. 检查路径是否存在。
4. 如果目标是目录，提示使用删除目录功能。
5. 调用 `ensure_recycle_bin()` 确保回收站存在。
6. 使用 `get_timestamp()` 生成时间戳。
7. 使用 `os.path.splitext()` 分离文件名和扩展名。
8. 生成回收站文件名。
9. 使用 `shutil.move()` 将文件移动到回收站。
10. 读取元数据列表，追加删除记录。
11. 保存元数据。

实现原理：

该函数通过“移动+记录元数据”实现软删除。只要回收站文件和元数据都存在，就可以在后续还原。

3.3.3 rename_file()

将文件或目录重命名。

实现流程:

1. 输入源路径。
2. 检查源路径是否存在。
3. 输入新文件名。
4. 检查新文件名是否为空。
5. 拼接目标路径。
6. 检查目标路径是否已存在。
7. 使用 `os.rename()` 执行重命名。

实现原理:

重命名本质上是在同一父目录下改变路径名称。函数通过 `os.path.dirname(src_path)` 获取源文件父目录，然后与新文件名拼接成目标路径。

3.3.4 `copy_move_file()`

实现文件或目录的复制和移动。

实现流程:

- step 1: 选择复制或移动
- step 2: 输入源路径
- step 3: 输入目标路径

复制逻辑:

1. 如果源路径是目录，使用 `shutil.copytree()` 递归复制。
2. 如果源路径是文件，使用 `shutil.copy2()` 复制文件。
3. 复制文件前使用 `os.makedirs()` 确保目标父目录存在。

移动逻辑:

1. 使用 `os.makedirs()` 确保目标父目录存在。
2. 使用 `shutil.move()` 移动文件或目录。

实现原理:

该函数通过 `mode` 变量保存用户选择，通过 `step` 变量控制流程。每一步都支持返回上一步，体现了状态机思想。

3.4 目录操作模块实现

3.4.1 `create_directory()`

创建一个目录。

实现流程：

1. 输入目录名。
2. 判断输入是否为空。
3. 解析绝对路径或相对路径。
4. 使用 `os.makedirs(path, exist_ok=True)` 创建目录。

实现原理：

`exist_ok=True` 表示如果目录已经存在，不会抛出异常，适合递归创建多级目录。

3.4.2 `delete_directory()`

删除目录，但同样采用回收站软删除。

实现流程：

1. 输入目录路径。
2. 检查路径是否存在。
3. 判断目标是否为目录。
4. 调用 `ensure_recycle_bin()`。
5. 生成时间戳目录名。
6. 使用 `shutil.move()` 将目录移动到回收站。
7. 记录原始路径、删除时间、回收站目录名。
8. 保存元数据。

实现原理：

目录删除与文件删除的思想相同，区别是目录没有扩展名处理，直接在目录名后拼接时间戳。

3.4.3 traverse_directory()

递归遍历目录，并以树形结构显示。

实现流程：

1. 输入要遍历的目录，回车默认当前目录。
2. 检查路径是否存在且是否为目录。
3. 使用 `os.walk()` 递归遍历。
4. 根据当前目录与根目录的层级差计算缩进。
5. 输出目录名和文件名。
6. 捕获 `KeyboardInterrupt`，允许用户中断遍历。

实现原理：

该函数通过路径分隔符数量计算目录层级，再使用“|”和“|——”组合出树形视觉效果。

3.5 搜索与统计模块实现

3.5.1 file_search()

支持按文件名搜索和按文件内容搜索。

实现流程：

- step 1: 选择搜索方式
- step 2: 输入搜索起始目录
- step 3: 输入文件名模式或内容关键词

按文件名搜索：

1. 用户输入通配符，例如 `*.txt`。
2. 使用 `os.walk()` 遍历目录。
3. 使用 `glob.fnmatch.fnmatch(f, pattern)` 判断文件名是否匹配。
4. 匹配成功则输出完整路径。

按内容搜索：

1. 用户输入关键词。

2. 使用 `os.walk()` 遍历目录中的所有文件。
3. 尝试以文本方式打开文件。
4. 使用 `enumerate(fh, 1)` 逐行读取并记录行号。
5. 如果关键词出现在某一行，输出文件路径、行号和内容片段。
6. 遇到权限错误或无法读取的文件时跳过。

实现原理：

该函数结合了递归遍历、通配符匹配、逐行文本搜索和异常处理，是项目中综合性较强的功能。

3.5.2 `file_statistics()`

统计文件或目录的基本信息。

对文件统计：

1. 显示绝对路径。
2. 显示文件类型。
3. 显示大小。
4. 显示创建时间、修改时间、访问时间。
5. 显示扩展名。
6. 如果是文本文件，统计行数。

对目录统计：

1. 使用 `os.walk()` 遍历目录。
2. 统计文件数量。
3. 统计子目录数量。
4. 累加所有文件大小。

实现原理：

该函数通过 `os.stat()` 获取单个路径元信息，通过 `os.walk()` 汇总目录内部信息，实现文件和目录的双模式统计。

3.6 回收站管理模块实现

3.6.1 `recycle_bin_menu()`

显示回收站子菜单，负责分发回收站相关操作。

菜单功能：

编号	功能
1	查看回收站
2	还原文件或目录
3	清空回收站
4	永久删除某个文件或目录
0	返回主菜单

实现原理：

该函数内部使用循环显示子菜单，并根据用户输入调用不同函数。如果子函数返回 `MAIN_MENU`，则立即退出子菜单并返回主菜单。

3.6.2 `list_recycle_bin()`

列出回收站中的所有记录。

实现流程：

1. 调用 `load_recycle_meta()` 读取元数据。
2. 如果列表为空，提示回收站为空。
3. 如果不为空，按照编号显示每条记录的回收站文件名、原始路径和删除时间。

实现原理：

该函数显示的是 JSON 元数据内容，而不是直接扫描回收站目录。这样可以展示原始路径等额外信息。

3.6.3 `restore_from_recycle()`

根据编号还原回收站中的文件或目录。

1. 实现流程：
2. 读取元数据。
3. 调用 `list_recycle_bin()` 显示所有项目。
4. 用户输入要还原的编号。
5. 检查编号是否合法。
6. 根据 `bin_name` 找到回收站中的实际文件。
7. 根据 `original_path` 找到原始路径。
8. 如果回收站文件不存在，提示错误。
9. 如果原始路径已经存在，拒绝还原，避免覆盖。
10. 使用 `os.makedirs()` 创建原始父目录。
11. 使用 `shutil.move()` 移回原路径。
12. 从元数据列表中删除该记录。
13. 保存元数据。

实现原理：

还原功能依赖 JSON 元数据完成路径恢复。程序不会猜测文件原位置，而是直接读取删除时保存的 `original_path`

3.6.4 `empty_recycle_bin()`

永久删除回收站中的所有内容。

实现流程：

1. 读取元数据。
2. 如果回收站为空，直接提示。
3. 要求用户输入 `yes` 二次确认。
4. 遍历所有元数据记录。
5. 如果回收站项目是目录，使用 `shutil.rmtree()` 删除。
6. 如果是文件，使用 `os.remove()` 删除。

7. 删除完成后保存空列表[]到元数据文件。

实现原理：

该函数是真正意义上的永久删除，因此加入了确认步骤，避免误操作。

3.6.5 permanent_delete_from_recycle()

永久删除回收站中的某一个项目。

实现流程：

1. 读取并显示回收站列表。
2. 用户输入要删除的编号。
3. 检查编号是否合法。
4. 要求用户输入 yes 确认。
5. 根据目标类型执行 `shutil.rmtree()` 或 `os.remove()`。
6. 从元数据列表中移除对应记录。
7. 保存更新后的元数据。

实现原理：

该函数相当于 `empty_recycle_bin()` 的单项版本，比清空回收站更灵活。用户可以只永久删除不需要恢复的某个文件，而保留其他文件。

3.7 主菜单调度模块实现

3.7.1 main()

程序主入口，负责显示菜单、接收用户选择、调用对应功能函数。

实现流程：

1. 使用 `while True` 持续显示主菜单。
2. 输出当前工作目录。
3. 输出 1 到 11 的功能选项。
4. 用户输入功能编号。
5. 使用 `if-elif` 分支调用对应函数。
6. 输入 0 时退出程序。

7. 捕获 `KeyboardInterrupt`，避免用户按 `Ctrl+C` 后程序直接崩溃。

功能分发：

用户输入	调用函数
1	<code>create_file()</code>
2	<code>delete_file()</code>
3	<code>rename_file()</code>
4	<code>copy_move_file()</code>
5	<code>create_directory()</code>
6	<code>delete_directory()</code>
7	<code>traverse_directory()</code>
8	<code>file_search()</code>
9	<code>file_statistics()</code>
10	<code>recycle_bin_menu()</code>
11	切换 <code>current_dir</code>
0	退出程序

实现原理：

`main()` 是整个程序的调度中心。各功能函数相对独立，执行完后返回主菜单，形成“主菜单分发-功能执行-返回主菜单”的循环结构。

3.8 学习心得

通过本次 Python 文件管理系统的开发，我们深入掌握了 Python 文件系统操作的核心技术，对 `os`、`shutil`、`json` 等模块的使用有了更加全面和深入的理解。与之前的面向对象实现相比，本次采用的面向过程架构让我们更加专注于流程控制和用户交互设计，体会到了不同编程范式在不同场景下的优势。

本次项目最大的突破是全局导航系统的实现。通过自定义 `BACK` 和 `MAIN_MENU` 对象，我们成功实现了跨所有功能模块的统一导航机制，让用户可以在任何操作步骤随时返回上一步或主菜单，极大提升了系统的易用性和容错性。这让我们认识到，一个优秀的命令行程序不仅要功能完善，更要有清晰的交互逻辑和人性化的操作体验。

回收站功能的增强是本次项目的另一个亮点。使用 `JSON` 格式替代传统纯文本文件存储元数据，不仅让数据结构更加清晰易读，也大大简化了元数据的增删

改查操作。新增的永久删除单个文件功能解决了传统回收站只能批量清空的痛点，让文件管理更加灵活。在实现过程中，我们深刻体会到了数据一致性的重要性，必须确保每一次文件操作都能同步更新元数据，避免出现数据不一致的问题。

文件内容搜索功能的实现让我们遇到了不少挑战。不同编码的文本文件、权限不足的文件和二进制文件都可能导致程序崩溃，通过添加完善的异常处理和编码忽略机制，我们最终实现了稳定可靠的内容搜索功能。这让我们明白，在处理外部资源时，必须充分考虑各种可能的异常情况，才能保证系统的健壮性。

此外，目录树可视化和详细的文件统计功能也让我们对 os 模块的底层操作有了更深入的了解。通过计算目录层级生成规范的树形结构，我们学会了如何将抽象的文件系统结构以直观的方式呈现给用户。

总的来说，本次项目是一次非常有价值的实践经历。它不仅让我们巩固了 Python 文件操作的基础知识，还提高了我们的系统设计能力、问题解决能力和代码规范意识。在未来的学习中，我们将继续探索 Python 在系统编程领域的应用，尝试实现多线程文件复制、文件压缩解压、批量重命名等更复杂的功能，进一步提升自己的编程水平。